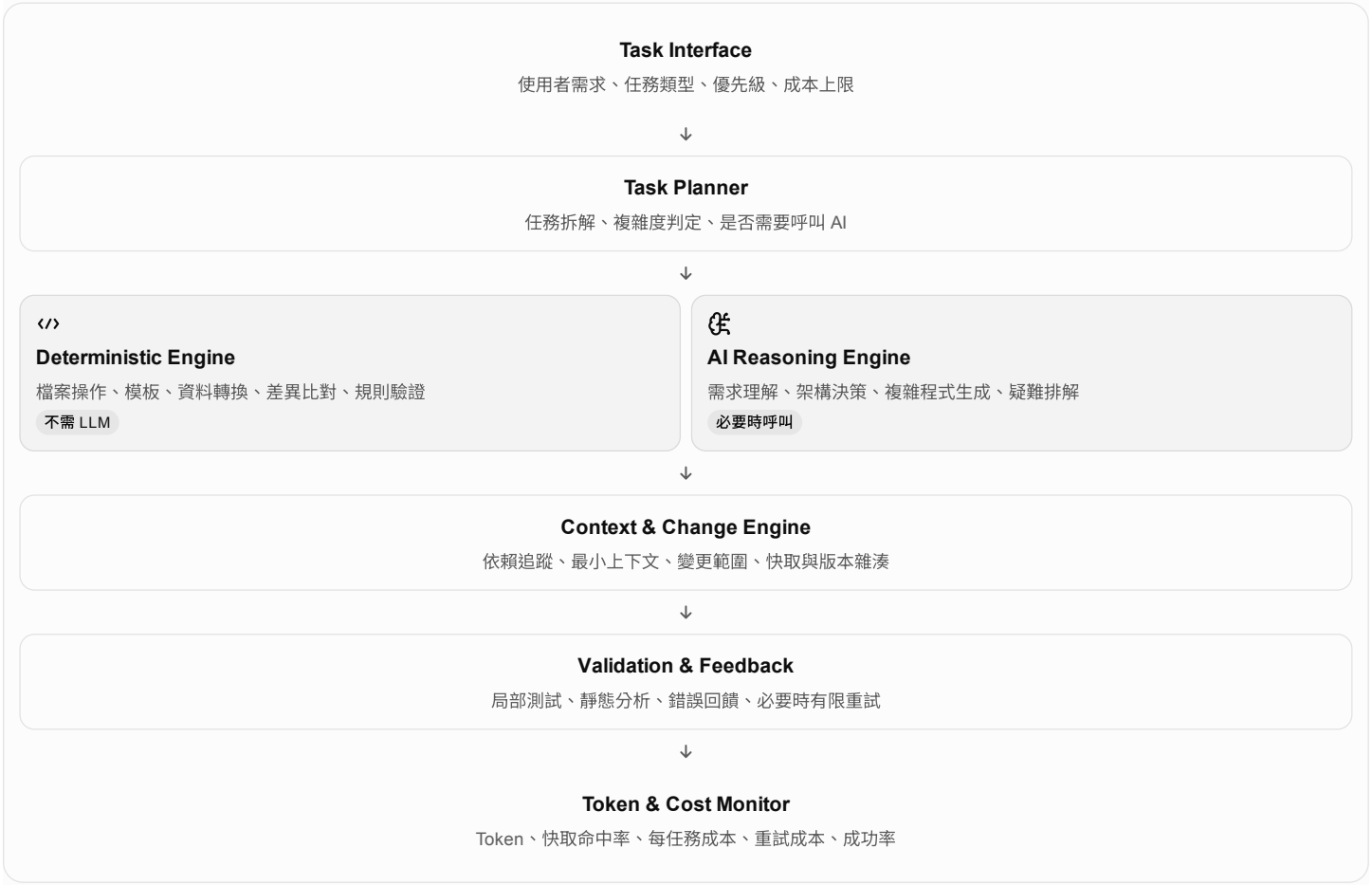
ChatGPT 說：

針對 SGGenAxis 的 **AI 自動開發工具**，如果主要目標是降低 Token 消耗與 API 成本，我建議不要只做「縮短 Prompt」，而是把整套工具設計成一個 **以結構化專案資料為核心、以差異更新為主、由程式負責能確定的工作、AI 只處理需要推理的部分** 的開發系統。

這個方向很適合你目前的 SGGenAxis 架構思路：先建立 3D／系統框架、資料更新模組，再透過命令層與 UI 提供操作。自動開發工具也應遵循相同原則，避免每次任務都讓 AI 重新理解整個專案。

一、建議的整體架構



這裡有個重要設計：**Token & Cost Monitor 應該是橫跨所有模組的監控層，而不是只有任務結束後才統計。** 它必須能在呼叫前預估成本、呼叫中檢查預算、呼叫後記錄實際用量。

二、最值得優先實作的 6 個功能

我會依照「成本改善潛力、開發難度、對產出品質的影響」排序，而不是一開始就建構複雜的多 Agent 系統。

P0 — 差異式上下文（Diff-based Context）

最高優先

每個任務只讀取目標檔案、必要的介面定義、直接依賴與相關測試。不要每次把整個專案送進模型。

關鍵機制：依賴圖、檔案索引、符號索引、變更範圍分析。

P0 — 確定性執行引擎（Deterministic Engine）

能透過程式完成的工作，不呼叫 AI，例如檔案複製、格式化、搜尋、模板套用、JSON Schema 驗證、雜湊比對與測試執行。

原則：AI 決定「該做什麼」，程式負責「可明確定義的執行步驟」。

P0 — 專案知識索引（Project Knowledge Index）

將模組、API、依賴、資料結構、規格與測試關係存成可查詢的結構化資料。AI 只收到與當前任務有關的結果，而不是整份設計文件。

第一版用 SQLite 加上檔案與符號索引即可，不需要先導入向量資料庫。

P1 — Prompt 與上下文快取

固定系統指令、輸出格式與工具定義的順序；把每次變動的任務資料放在後面，讓重複前綴更容易命中 API Prompt Cache。

OpenAI 支援的模型可使用 Prompt Caching，但命中率會受前綴、模型、快取有效時間與請求路由影響，不能假設每次都命中。

OpenAI API

P1 — 模型路由（Model Routing）

簡單分類、欄位抽取、局部小修改使用低成本模型或規則；跨模組架構設計、複雜除錯才升級到高能力模型。

路由依據應包括任務類型、預估 Token、錯誤風險與歷史成功率，而不是只看任務字數。

P1 — 驗證回饋與重試控制

先用編譯器、測試與靜態分析定位問題，只把錯誤摘要及相關程式碼回傳給 AI。設定重試上限，避免反覆生成完整檔案。

不能只追求少 Token；如果修改失敗、反覆重做，總成本反而更高。

三、如何實際減少 Token

我建議同時使用兩種策略，但不要把它們混為一談。

策略	減少實際傳送的內容	降低輸入計費	降低輸出計費
局部上下文	是	通常可以	間接有幫助
差異式程式碼輸出	是	有時	是
Prompt Caching	不一定	可降低符合條件的快取輸入成本	否
確定性程式處理	可避免部分 AI 呼叫	可以	可以
低成本模型路由	不一定	可能	可能
減少無效重試	是	是	是

OpenAI 的 Prompt Caching 是重用相同前綴的已處理狀態，不代表模型不需要接收新任務資料，也不會直接減少生成答案的 Token。

OpenAI API

因此，SGGenAxis 的第一目標應是減少不必要的 AI 呼叫及上下文，再利用快取降低剩餘呼叫的成本。

四、建議的實作方案

建議分成三個階段。第一階段先讓現有原型省 Token，不必先重寫整個自動開發工具。

Phase 1
基礎控制

建立 Token 成本基線

- 記錄每次 API 呼叫的輸入、輸出與快取 Token。
- 記錄模型、任務類型、成功／失敗、重試次數與實際費用。
- 加入單次任務 Token 預算及費用上限。
- 將固定 Prompt、任務資料與專案上下文分開管理。

完成標準：能回答「哪種任務最花錢、哪些呼叫沒有產生有效成果」。

Phase 2
降低消耗

導入局部上下文與差異更新

- 建立專案檔案索引及模組依賴圖。
- 將任務轉換為結構化指令。
- 只傳送目標檔案片段、相關介面與必要規格。
- 要求 AI 回傳指定格式的修改結果或 Patch。
- 由本地程式套用變更，再執行局部驗證。

完成標準：一般小修改不需要重新讀取完整專案，也不會因上下文不足而破壞相鄰功能。

Phase 3
自動最佳化

導入模型路由、快取與品質評估

- 依任務風險選擇模型。
- 對重複且穩定的上下文提高快取命中率。
- 根據錯誤類型決定是否重試或升級模型。
- 比較不同策略的總成本與任務成功率。

完成標準：以實際成功任務的平均成本作為最佳化依據，而不是只比較 Token 數量。

五、成本監控應該如何設計？

建議每次 AI 呼叫都記錄一筆 `LLMCallRecord`，例如：

JSON

```
{
  "taskId": "task_001",
  "callId": "call_003",
  "purpose": "code_generation",
  "model": "configured_model",
  "inputTokens": 4200,
  "cachedInputTokens": 3000,
  "outputTokens": 650,
  "retryIndex": 0,
  "success": true,
  "latencyMs": 2400,
  "estimatedCost": 0.002
}
```

以上數字只是資料格式示例，並非實際 API 計價結果。正式系統應依 API 回傳的用量、模型價格與幣別計算費用，並記錄快取寫入等適用項目。

至少追蹤以下指標：

指標	用途
Cost per Successful Task	每個成功任務的平均成本
Input Tokens per Task	每個任務輸入 Token
Output Tokens per Task	每個任務輸出 Token
Cache Hit Rate	快取輸入占比
Retry Cost Ratio	重試消耗占總成本的比例
First-pass Success Rate	第一次執行就成功的比例
Validation Failure Rate	生成結果未通過驗證的比例

其中最重要的是 **Cost per Successful Task**。例如某個策略把 Token 減少 50%，卻讓失敗率大幅提高，最終每個成功任務的成本可能反而增加。

六、如何估算改善幅度？

實際節省比例必須由你的原型測量，現在還沒有足夠的用量資料能準確預測。我建議先用一個可調整的模型試算：

API 成本試算器

請輸入假設值。此工具只計算成本情境，不代表預測或實測結果。

每月任務數 **1,000**

100 5,000 10,000

每個任務原始 API 成本（USD） **$0.020**

預期原始 API 成本降低幅度 **40%**

0% 40% 80%

新增系統成本占原始 API 成本比例 **5%**

0% 10% 20%

原始每月 API 成本

$20.00

最佳化後每月成本

$12.60

每月淨節省

$7.40

淨節省比例

37.0%

複製試算結果

這裡假設任務數與品質不變、成本降低幅度已包含 Token 與模型路由帶來的效果，而且新增成本以原始 API 成本為基準。它不包含品質退化、任務重做及不同任務複雜度的差異，所以正式決策仍要依照實測結果。

七、我的最終建議

對 SGGenAxis 而言，我會採取以下順序：

1. **先建立 Token／成本追蹤。** 沒有基線，就不知道錢花在哪裡。
2. **再做 Context Manager 與差異式更新。** 這是降低重複輸入最直接的架構改善。
3. **將確定性工作從 AI 流程中移除。** 不讓模型做一般程式就能完成的工作。
4. **最後再導入模型路由與 Prompt Caching。** 讓剩下的 AI 呼叫變得更便宜。

另外，不建議一開始就使用多 Agent 協作來追求效率。多個 Agent 可能重複讀取上下文、彼此傳遞大量文字，還會增加呼叫次數。只有在任務可以有效平行化，而且實測證明總成本與完成時間更好時，才值得加入。

下一步最有價值的工作，是盤點你現有自動開發工具的執行流程。 如果你提供目前的流程圖、程式碼架構或一筆實際 API 請求與回應用量，我就能把上述方案落到具體模組、資料結構、呼叫流程及優先修改清單，而不只是停留在概念設計。